

AASTool Frontend Architecture

AASTool — Frontend Architecture & Component Documentation

Enterprise Edition | Version 1.0.0

Table of Contents

- 1. [Technology Stack](#)
- 2. [Project Structure](#)
- 3. [Architecture Overview](#)
- 4. [Application Routes](#)
- 5. [Component Catalog](#)
- 6. [Data Flow](#)
- 7. [State Management](#)
- 8. [Styling & Theming](#)
- 9. [PWA Features](#)
- 10. [Performance Optimizations](#)
- 11. [Error Handling](#)
- 12. [Build Configuration](#)

Technology Stack

Technology	Version	Purpose
Next.js	16.2.6	React framework (App Router, Turbopack)
React	19.2.6	UI library
TypeScript	5.9.3	Type safety
Tailwind CSS	4.x	Utility-first CSS framework
TanStack React Query	5.x	Server state management & caching

Technology	Version	Purpose
Axios	1.5.x	HTTP client
Framer Motion	12.40.x	Animations & transitions

Module Resolution

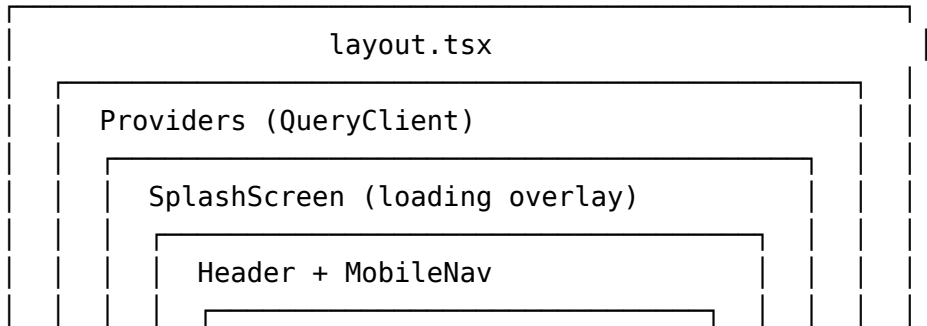
- moduleResolution: "bundler" — modern bundler-style resolution
- @/ path alias — maps to project root (e.g., @/components/Header → ./components/Header)

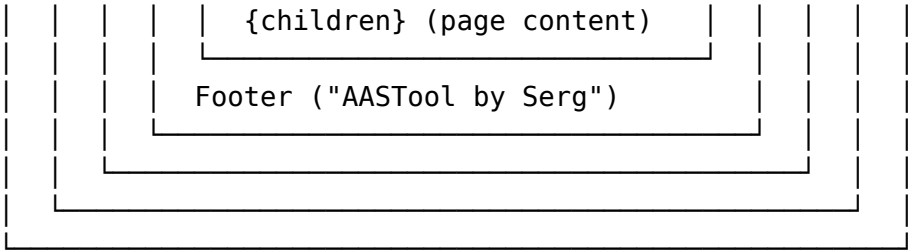
Project Structure

frontend/	
├─ package.json	# Dependencies & scripts
├─ tsconfig.json	# TypeScript config (bundler resolution)
├─ next.config.js	# Next.js config
├─ tailwind.config.ts	# Tailwind CSS 4 config
├─ postcss.config.js	# PostCSS plugins
├─ public/	
│ └─ manifest.json	# PWA manifest
│ └─ sw.js	# Service worker
├─ app/	# Next.js App Router pages
│ └─ layout.tsx	# Root layout (providers, splash, footer)
│ └─ globals.css	# Global styles + Tailwind directives
│ └─ page.tsx	# Home/landing page
│ └─ dashboard/	
│ └─ page.tsx	# Main dashboard
│ └─ building/	
│ └─ page.tsx	# Building types overview
│ └─ buildings/	
│ └─ page.tsx	# Buildings list
│ └─ [id]/	
│ └─ page.tsx	# Building detail (dynamic)
│ └─ criteria/	
│ └─ page.tsx	# Criteria reference table
│ └─ certification/	
│ └─ page.tsx	# Certification engine
│ └─ dimensions/	
│ └─ page.tsx	# Assessment dimensions
│ └─ disability/	
│ └─ page.tsx	# Disability types

reports/	
└─ page.tsx	# Generated reports
login/	
└─ page.tsx	# Login page
assessments/	
└─ new/	
└─ page.tsx	# Create new assessment
└─ [id]/	
└─ page.tsx	# Assessment detail (dynamic)
components/	# Shared React components
└─ Header.tsx	# Top navigation bar
└─ BottomNav.tsx	# Mobile bottom navigation
└─ BuildingCard.tsx	# Building summary card
└─ ErrorBoundary.tsx	# React error boundary
└─ OfflineIndicator.tsx	# Network offline banner
└─ PWAProvider.tsx	# PWA install prompt
└─ SplashScreen.tsx	# SERG branded loading screen
└─ layout/	
└─ mobile-nav.tsx	# Mobile nav + theme context
└─ navbar.tsx	# Desktop sidebar navigation
lib/	# Shared utilities
└─ api/	
└─ client.ts	# Axios instance + interceptors
└─ endpoints.ts	# All API endpoint functions
└─ hooks/	
└─ useBuildings.ts	# React Query: buildings
└─ useCriteria.ts	# React Query: criteria
└─ useAssessment.ts	# React Query: assessment by ID
└─ queryClient.ts	# React Query client
configuration	└─ theme.ts
tokens + gauge helpers	# Color/gradient
	└─ storage.ts
helpers	# localStorage persistence
animation presets	# Framer Motion
└─ pwa.ts	# PWA registration utilities

Architecture Overview





Data Flow (Unidirectional)

User Action → Page Component → React Query Hook → endpoints.ts →
Axios Client → REST API

↓
User sees UI ← React re-render ← Cache update ← JSON Response
← Backend Server

All data fetching flows through these layers: 1. **Page component** renders and calls a React Query hook 2. **Hook** (useBuildings, useCriteria, useAssessment) wraps an endpoint call 3. **Endpoint function** calls api.get() / api.post() and unwraps the response 4. **Axios instance** adds base URL, timeout, and error normalization 5. **React Query cache** stores the result, provides loading/error states

Application Routes

Static Routes (12 pages)

Route	Page	Purpose	Data Dependencies
/	page.tsx	Landing page with hero, quick links	None (static)
/dashboard	dashboard/page.tsx	Main dashboard with KPIs	useBuildings, useCriteria
/building	building/page.tsx	Building types overview	useBuildings
/buildings	buildings/page.tsx	All buildings with assessment counts	useBuildings, useAssessment
/criteria	criteria/page.tsx	Full criteria table with	useCriteria

Route	Page	Purpose	Data D
		search/filter	
/certification	certification/page.tsx	NEB certification engine	useBuildin endpoint
/dimensions	dimensions/page.tsx	Assessment dimensions (AD1-AD5)	API: getAssessm
/disability	disability/page.tsx	Disability types (DT1-DT5)	API: getDis
/reports	reports/page.tsx	Generated evaluation reports	API: getRep
/login	login/page.tsx	Login form	API: POST ,
/assessments/new	assessments/new/page.tsx	Create new assessment	useBuildin createAsse

Dynamic Routes (2 pages)

Route Pattern	Page	Purpose	URL Params
/buildings/[id]	buildings/[id]/page.tsx	Building detail & assessment history	id: string (UUID)
/assessments/[id]	assessments/[id]/page.tsx	Assessment result detail	id: string (UUID)

Route Mapping

/	→ Home landing
/dashboard	→ Dashboard KPIs
/building	→ Building types grid
/buildings	→ Buildings list
/buildings/:id	→ Building detail
/criteria	→ Criteria reference
/certification	→ Certification tool
/dimensions	→ AD dimensions
/disability	→ DT types
/reports	→ Reports list
/login	→ Authentication
/assessments/new	→ New assessment form
/assessments/:id	→ Assessment result

Component Catalog

Layout Components

layout.tsx (Root Layout)

Purpose: Every page's outer shell — provides QueryClient, theme context, splash screen, header, mobile nav, and global footer.

Structure:

```
<html>
  <body>
    <Providers>                // React Query Provider
    <SplashScreen />           // SERG branded loading overlay
    <Header />                  // Top nav bar
    <MobileNav />               // Bottom mobile nav
    {children}                  // Page content
    <footer>                    // Global footer
      AASTool by Serg | Dev by Y
    </footer>
  </Providers>
</body>
</html>
```

Key imports: @/components/Header, @/components/layout/mobile-nav, @/components/SplashScreen, @/components/Providers

Header.tsx

Purpose: Top navigation bar with AASTool branding and theme toggle.

Features: - “AASTool” site title with “by Serg | Dev by Y” tagline - Theme toggle (light/dark) via useTheme hook from mobile-nav context - ARIA labels: aria-label="Toggle theme" - Responsive: adapts to screen width

navbar.tsx (Desktop Sidebar)

Purpose: Desktop-only sidebar navigation with icon links.

Features: - Navigation links to main sections - Active state highlighting - Icon-based navigation items

mobile-nav.tsx (Mobile Bottom Nav + Theme Provider)

Purpose: Mobile bottom tab bar AND theme context provider.

Features: - Bottom navigation tabs: Dashboard, Buildings, Criteria, Reports, Settings - Theme context: ThemeProvider with useTheme() hook - Active tab indicator - CSS transitions for tab changes

Functional Components

SplashScreen.tsx

Purpose: SERG-branded loading screen shown on initial page load.

Behavior: - Full-screen dark overlay with subtle animated background glow - “SERG” in gradient text (accent colors) - Animated shimmer bar (slides left-to-right) - “Smart Engineering Research Group” subtitle - Fades out at **1.8 seconds**, fully removed at **2.3 seconds** - Client component: uses useState / useEffect

Animation:

```
@keyframes slide {
  0% { transform: translateX(-100%); }
  100% { transform: translateX(400%); }
}
--animate-slide: slide 2s ease-in-out infinite;
```

BuildingCard.tsx

Purpose: Displays a building type summary as a card.

Props: | Prop | Type | Description | |—|—|—|—| | building | string | Building type name (fetched from API — dynamic, not hardcoded) | | onClick | () => void | Click handler |

States: Loading skeleton, populated card, error state.

ErrorBoundary.tsx

Purpose: React error boundary — catches render errors in child tree.

Features: - Displays fallback UI on error - “Something went wrong” message with retry button - Prevents blank white screens on runtime errors

OfflineIndicator.tsx

Purpose: Banner shown when the browser goes offline.

Features: - Listens to online/offline window events - Slides in from top when offline - “No internet connection” warning - Auto-hides when connection restored

PWAProvider.tsx

Purpose: Provides PWA install prompt functionality.

Features: - Listens for beforeinstallprompt event - Shows install button when available - Manages installPrompt state

Data Flow

API Client Layer

client.ts (Axios Instance)

```
const api = axios.create({
  baseURL: process.env.NEXT_PUBLIC_API_URL ||
"http://localhost:4000/api/v1",
  timeout: 15000,
  headers: { "Content-Type": "application/json" }
});
```

Response Interceptor: - Normalizes error responses into { status, message, ...data } format - Handles network errors (status 0) - Preserves Axios error shape for React Query error handling

Error Shape (after interceptor):

```
{
  status: number;           // HTTP status code (0 for network
errors)
  ok?: boolean;             // false on error
  error?: string;           // Error message from backend
  message?: string;         // Fallback error message
}
```

endpoints.ts (API Functions)

All API functions grouped in a single endpoints object:

Function	Method	URL	Returns
healthCheck()	GET	/health	Health status
getBuildings()	GET	/buildings	Building array
getBuilding(id)	GET	/buildings/:id	Building detail

Function	Method	URL	Returns
createAssessment(payload)	POST	/assessments	Created assessment
getAssessment(id)	GET	/assessments/:id	Assessment result
evaluate(payload)	POST	/evaluate	Evaluation result
getCriteria()	GET	/criteria	Criterion array
createCriterion(payload)	POST	/criteria	Created criterion
getBuildingTypes()	GET	/building-types	Building types
getAssessmentDimensions()	GET	/assessment-dimensions	AD array
getDisabilityTypes()	GET	/disability-types	DT array
getNebThresholds()	GET	/neb-thresholds	NEB bands
getReports()	GET	/reports	Report array

Response Unwrapping: The `unwrap<T>()` helper extracts `response.data.data` (or falls back to `response.data`). The `evaluate` endpoint is a special case — it extracts `res.data.result` directly, since the `evaluate` API returns `{ ok: true, result: {...} }` rather than `{ ok: true, data: {...} }`.

React Query Hooks

useBuildings.ts

```
export function useBuildings() {
  return useQuery({
    queryKey: ['buildings'],
    queryFn: () => endpoints.getBuildings(),
  });
}
```

Returns: { data, isLoading, isError, error }

useCriteria.ts

```
export function useCriteria() {
  return useQuery<Criterion[]>({
    queryKey: ['criteria'],
    queryFn: () => endpoints.getCriteria(),
  });
}
```

```
    });  
  }
```

useAssessment.ts

```
export function useAssessment(id: string) {  
  return useQuery({  
    queryKey: ['assessment', id],  
    queryFn: () => endpoints.getAssessment(id),  
    enabled: !!id, // Only fetch when id is truthy  
  });  
}
```

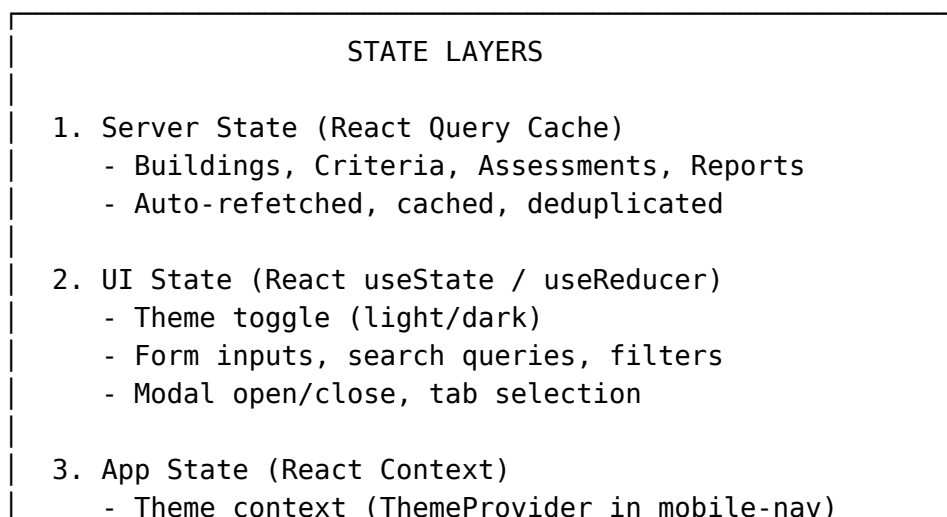
Query Client Configuration

```
// lib/queryClient.ts  
new QueryClient({  
  defaultOptions: {  
    queries: {  
      staleTime: 5 * 60 * 1000, // 5 minutes  
      retry: 2,  
      refetchOnWindowFocus: false,  
    },  
  },  
});
```

Cache Strategy: - **staleTime:** 5 minutes — data considered fresh for 5 min before refetch - **retry:** 2 attempts on failure - **refetchOnWindowFocus:** disabled to reduce server load

State Management

Architecture



- No global app state needed (data in React Query)
- 4. Client Persistence (localStorage)
 - Saves certification scores, dashboard state
 - Managed via lib/storage.ts helpers
 - Keys: STORAGE_KEY, CERTIFICATION_KEY

Theme System

The theme toggle is implemented via React Context in mobile-nav.tsx:

```
const ThemeContext = createContext<{
  theme: 'light' | 'dark';
  toggleTheme: () => void;
}>(...);
```

Components consume it via:

```
const { theme, toggleTheme } = useTheme();
```

Theme is applied via CSS custom properties and Tailwind's dark: variant. No CSS-in-JS runtime.

Styling & Theming

Design Tokens (lib/theme.ts)

Centralized color and gradient tokens used across the app:

```
// Disability Type colors (DT1-DT5)
export const DT_COLORS: Record<string, string> = { ... };
// Assessment Dimension colors (AD1-AD5)
export const AD_COLORS: Record<string, string> = { ... };
// Dashboard gradient tokens
export const DT_GRADIENT: string = '...';
export const AD_GRADIENT: string = '...';
// OBS gauge dynamic colors
export function getObsGaugeColors(obs: number): { color1:
string; color2: string };
// NEB badge gradient per class
export function getNebBadgeGradient(nebClass: string):
string;
```

Tailwind CSS 4 Configuration

```
// tailwind.config.ts
export default {
  content: ['./app/**/*.{ts,tsx}', './components/**/*.{ts,tsx}'],
  darkMode: 'class',
  theme: {
    extend: {
      colors: {
        primary: { ... },
        accent: { ... },
      },
    },
  },
};
```

Custom Animations

Defined in globals.css:

```
@theme {
  --animate-slide: slide 2s ease-in-out infinite;
}

@keyframes slide {
  0% { transform: translateX(-100%); }
  100% { transform: translateX(400%); }
}
```

Framer Motion Presets

Defined in lib/animations.ts: - fadeIn — opacity 0 → 1 transition -
slideUp — translateY + opacity entrance - scaleIn — scale entrance
for cards/modals

PWA Features

Manifest (public/manifest.json)

```
{
  "name": "AASTool",
  "short_name": "AASTool",
  "start_url": "/",
  "display": "standalone",
  "background_color": "#ffffff",
```

```
"theme_color": "#000000"  
}
```

Service Worker (public/sw.js)

- Caches critical assets for offline access
- Network-first strategy for API calls
- Cache-first strategy for static assets

PWA Registration (lib/pwa.ts)

Registers the service worker and handles updates: -
registerServiceWorker() — registers sw.js - Handles updatefound event for new version notifications

Performance Optimizations

Technique	Implementation
Code splitting	Next.js automatic route-based splitting
Server state caching	React Query with 5-min stale time
Lazy loading	Dynamic imports for non-critical components
Image optimization	Next.js <Image> with WebP
CSS optimization	Tailwind JIT (Just-in-Time) compilation
HTTP caching	ETags, Cache-Control headers from backend
Bundle analysis	next build --analyze

Error Handling

Hierarchy

Error Boundary (component crashes)
└─ React Query error states (API failures)
 └─ Axios interceptor (network errors, HTTP errors)
 └─ Component-level try/catch (form submission)

Error Boundary

- Wraps the entire app in layout.tsx
- Catches unhandled render errors
- Shows fallback UI with retry button

- Prevents blank white screen

API Error Handling

The Axios interceptor normalizes all errors into:

```
{ status: number; error?: string; message?: string }
```

React Query exposes `isError` and `error` on every hook result, so pages can conditionally render error states.

Offline Handling

- `OfflineIndicator` shows a banner when `navigator.onLine` is false
 - React Query can be configured to pause queries when offline
 - Service worker provides cached fallbacks
-

Build Configuration

Scripts (package.json)

```
{
  "dev": "next dev",          // Development server
  (Turbopack)
  "build": "next build",      // Production build
  "start": "next start",      // Production server
  "lint": "next lint"         // ESLint
}
```

TypeScript (tsconfig.json)

Key settings:

```
{
  "compilerOptions": {
    "moduleResolution": "bundler",
    "baseUrl": ".",
    "paths": {
      "@/*": ["./*"]
    },
    "strict": true,
    "jsx": "react-jsx"
  }
}
```

Next.js Config (next.config.js)

- App Router enabled by default (Next.js 16)
- Turbopack for development builds
- TypeScript path aliases configured via tsconfig.json

AASTool by Serg | Dev by Y